LoRA: Low-Rank Adaptation of Large Language Models

1. 연구 배경 (Problem Background)

최근 NLP에서는 대규모 사전학습 모델(LLM)을 다양한 downstream task에 맞게 fine-tuning하여 사용하는 방식이 일반적이다. 그러나 모델 규모가 커질수록 기존 fine-tuning 방식은 다음과 같은 문제가 발생한다.

GPT-3(175B) 같은 모델은 모든 파라미터를 업데이트해야 하므로 비용이 매우 큼

task마다 전체 모델 복사본을 저장해야 하므로 storage 비용 증가

학습 시 GPU memory 요구량이 매우 높음

즉, 대형 모델의 파라미터 전체를 업데이트하는 방식은 비효율적이다.

이를 해결하기 위해 parameter-efficient fine-tuning 연구가 등장하였다.

대표 방법

- Adapter
- Prefix tuning
- Prompt tuning

하지만 기존 방법은

- 추론 latency 증가
- 성능 저하

문제가 존재한다.

2. 핵심 아이디어 (Core Idea)

논문의 핵심 가설은 다음과 같다.

대형 모델을 특정 task에 맞게 adaptation할 때 필요한 weight update는 low-rank 구조를 가진다

즉, 실제로 필요한 파라미터 변화는 전체 weight space가 아니라 작은 subspace에 존재한다는 것이다.

이를 기반으로 LoRA (Low-Rank Adaptation) 방법이 제안된다.

3. LoRA 방법

기존 Transformer layer의 weight matrix:

$$W_0 \in \mathbb{R}^{d \times k}$$

fine-tuning에서는

$$W = W_0 + \Delta W$$

LoRA는 업데이트를 다음과 같이 분해한다.

$$\Delta W = BA$$

4. Transformer에서의 적용

LoRA는 Transformer의 여러 weight에 적용할 수 있다.

대표적으로 self-attention weight

논문에서는 주로 다음 두 weight에 적용하였다.

- Query projection
- Value projection

이렇게 하면 trainable parameter 수를 크게 줄일 수 있다.

5. 장점

1. 파라미터 감소

GPT-3 175B 기준 trainable parameter 10,000배 감소

2. 메모리 절약

optimizer state를 저장할 필요가 없기 때문에 GPU memory 약 3배 감소

3. inference latency 없음

LoRA weight는 추론 시

$$W = W_0 + BA$$

로 merge할 수 있다. 따라서 adapter와 달리 추가 계산이 없다.

4. task switching 쉬움

기본 모델은 공유하고 LoRA weight만 교체하면 task 변경 가능

6. 실험 결과

다양한 모델에서 실험 수행

- RoBERTa
- DeBERTa
- GPT-2
- GPT-3

결과

LoRA는 full fine-tuning과 동일하거나 더 높은 성능을 보였다.

훨씬 적은 파라미터로 SOTA 수준 성능을 달성했다.

Retrieval-Augmented Generation (RAG)

1. 연구 배경

대형 언어모델은 많은 지식을 모델 파라미터에 저장한다.

하지만 다음과 같은 한계가 있다.

- 최신 정보 업데이트 어려움
- 지식 출처 추적 불가
- hallucination 문제

즉 LM은 knowledge retrieval 능력이 부족하다.

이를 해결하기 위해

parametric memory + external memory

를 결합한 모델이 제안되었다.

2. 핵심 아이디어

RAG의 핵심 아이디어는 생성 모델이 외부 문서 검색(retrieval)을 수행하고

검색된 문서를 기반으로 답을 생성한다. 즉 LLM + 검색 시스템을 결합한 구조이다.

3. RAG 구조

RAG는 두 개의 주요 컴포넌트로 구성된다.

1. Retriever

$$p_\eta(z|x)$$

입력 x를 기반으로 관련 문서 z를검색

사용 모델

- Dense Passage Retriever (DPR)

retrieval 방식

- vector similarity
- Maximum Inner Product Search

2. Generator

$$p_\theta(y|x, z)$$

입력

- query x
- retrieved document z

사용 모델

- BART seq2seq transformer

즉 input + retrieved document → generator → answer

4. RAG 모델 종류

논문에서는 두 가지 모델을 제안했다.

1. RAG-Sequence

하나의 document로 전체 output 생성

$$p(y|x) = \sum_z p(z|x)p(y|x,z)$$

동일 document 사용

2. RAG-Token

token마다 다른 document 사용 가능

$$p(y|x) = \prod_i \sum_z p(z|x)p(y_i|x,z,y_{<i})$$

여러 document 정보 결합 가능

5. 학습 방법

retriever와 generator를 end-to-end로 학습한다.

Loss

$$-\log p(y|x)$$

즉 retrieval, generation 둘 다 동시에 최적화한다.

6. 실험 결과

다양한 knowledge-intensive task에서 실험 수행

대표 task

- Open-domain QA
- Fact verification
- Question generation

결과

- 기존 모델보다 SOTA 성능 달성
- 특히 open-domain QA에서 큰 성능 향상

또한

- 더 factual
- 더 specific
- hallucination 감소

효과가 나타났다.